ESCOLA DE MATEMÁTICA APLICADA

Machine Learning

LAB EXERCISE 1

This lab exercise must be submitted by April 3th, 2026 at 10:00 am.
Late submissions will not be accepted and will be marked as zero.

This lab exercise is mandatory and could be assessed.
It is worth 1% of your total final grade for this course.

Submission Instructions

Submit your file "**lab1_123456.ipynb**" through EClass (replace "123456" for your unique 6-digit identifier you selected at the beginning of the course). The files you submit cannot be read by any other students. You can replace your submission as many times as you like by resubmitting it, although only the last version sent is kept.

If you have last-minute problems with your EClass submission, email your assignment as an attachment to alberto.paccanaro@fgv.br with the subject "URGENT – LAB 1 SUBMISSION". In the body of the message, explain the reason for not submitting it through EClass.

For any questions about the laboratory, please send an email to *santiago.ferreyra@fgv.br*

All the work you submit should be solely your own work.
Coursework submissions will be checked for this.

In this lab, you will be implementing the **KNN algorithm** that we saw in class by modifying the provided 'lab1_2026.1.ipynb' notebook file.

You will be applying the KNN algorithm on the **Iris dataset** to classify flowers into classes. Each of the 150 flowers of the dataset is described by the following features:

- Sepal length in cm
- Sepal width in cm
- Petal length in cm
- Petal width in cm

Sklearn.datasets.load_iris(return_X_y = True) is a sklearn function that provides a 150x4 matrix, where each column is a flower, and each row contains one of the features described above (in that order). It also contains a vector with a label for each of the flowers.

You will implement a program in Python to:

- **Load the dataset:** sklearn.datasets.load_iris(return_X_y = True)

- **Choose two features to plot the points in 2D, then:**

  - Plot the points and the distribution of the data, make sure you:
    - Assign a color to each class.
    - Paint the points and its distribution with the color of the class they belong (use a circle to represent the points and a continuous curve for the density estimation).
    - Label the axis with the name of the corresponding feature.

  - Do this for all six combination of features pairs, and see which one separates the class better in the plots. You can use either matplotlib or seaborn libraries.

- **Implement the *k*-nearest neighbours algorithm.** For a given point *x* you need to:
  - Find the *k* nearest neighbours of *x*.
  - Count how many times each class appears between the *k* neighbours of *x*.
  - Assign *x* to the class with the highest count of neighbours.

Today you should aim at finishing the plots for the dataset, and implementing the k-nearest neighbours algorithm in a function within the provided notebook.

<u>During next week you should:</u>

- Divide your dataset into training, testing and validation sets. (You can use sklearn.model_selection.train_test_split() function)
- Using the training and validation sets, test your defined distances and different values of *k* nearest neighbors.
- Measure the accuracy of the model on the validation and testing dataset, remember that:
    - The accuracy can be measured for each class, as well as the overall accuracy.
    - A prediction is considered to be correct for a given sample if the predicted class is the same as the true class of the sample.
    - The overall accuracy can be defined as the number of correct predictions, divided by the total amount of samples of the test set.
    - The class accuracy can be defined as the number of correct predictions for that class, divided by the total amount of samples of the test set belonging to that class.
- Make bar plots for the overall accuracy and the accuracy for each class.

- Make a plot for the training/validation and testing sets:
    - Plot the training points using a circle, and paint them with the color of the class they belong.
    - Plot the points on the testing set using an *X* and paint them with the color of the class they were assigned by the KNN algorithm.
    - Add labels and titles.

- Check your results of K and accuracy using `KNeighborsClassifier` from `sklearn`.
- Think about the questions at the end of the lab.

Have fun 😊